Parnassus: A GPU-enabled, Python-based Package for Fast Particle Detector Simulation and Reconstruction

Etienne Dreyer[a], Abdelrahman Elabd[b], Eilam Gross[a], Dmitrii Kobylianskii[a], Benjamin Nachman[c,d]

[a] *Department of Particle Physics & Astrophysics, Weizmann Institute of Science, Rehovot, Israel*
[b] *Department of Physics, University of Washington, Seattle, WA 98195, USA*
[c] *Department of Particle Physics and Astrophysics, Stanford University, Stanford, CA 94305, USA*
[d] *Fundamental Physics Directorate, SLAC National Accelerator Laboratory, Menlo Park, CA 94025, USA*

Abstract

We present the public software release of Parnassus, a pure-Python, GPU-compatible framework for fast detector simulation and reconstruction in particle and nuclear physics. Parnassus provides a user friendly surrogate model of computationally expensive Geant4-based detector simulation and reconstruction chains with interchangeable detector models. This initial release includes two models of the CMS detector - one based on a flow-matching neural network architecture and one based on a PyTorch implementation of the Delphes (parametric bias and smearing) CMS card. A PyTorch version of the ATLAS Delphes model and a neural network model of the ALEPH detector are also available. All detector-specific backends share the same process-agnostic and detector-agnostic API: users select a detector card—analogous to choosing a detector card in Delphes—and the same tool can be applied to arbitrary physics processes without retraining. There are native interfaces to the event generator Pythia and event clustering package fastjet. Unlike previous C++/ROOT-based tools, Parnassus runs natively on GPU and requires no ROOT installation. We describe the installation, command-line and Python API, configuration system, and demonstrate the framework on Standard Model and BSM processes.

Keywords: fast simulation, event reconstruction, detector simulation, particle flow, generative models, flow matching, diffusion models, particle physics, nuclear physics

PROGRAM SUMMARY

Program title:	Parnassus
Developer's repository link:	https://github.com/parnassus-hep/parnassus
Licensing provisions:	MIT
Programming language:	Python ($\geq$ 3.12)
Nature of problem:	Full detector simulation and reconstruction based on Geant4 is computationally expensive, often dominating the computing budget of large-scale particle and nuclear physics experiments and is computationally prohibitive for individual researchers. A fast, accurate surrogate is needed that maps truth-level (generator-level) particles directly to detector-level reconstructed particles, preserving the fidelity of the full chain while being orders of magnitude faster.

Solution method: Parnassus offers interchangeable detector backends sharing a common API. These backends include both neural network models (based on flow matching) as well as parameterized approaches (TorchDelphes, a re-implementation of select Delphes [26] detector cards). The former has been validated in previous papers [2, 3] and the latter is validated in this work against the C++ Delphes 3.5.1 cards for the CMS and ATLAS detectors. All backends are followed by physics-based post-processing (jet clustering via FastJet [18], lepton isolation). Users select a detector card—analogous to a Delphes card—to target a specific experiment.

Additional comments including restrictions and unusual features: The framework is implemented entirely in Python with PyTorch and runs natively on CPU and GPU. It has no dependency on ROOT; output to ROOT format is optionally handled via `uproot` [25]. The current release ships with CMS and ATLAS parametric cards and CMS and ALEPH neural models; additional detector models for other experiments will be added in future releases.

1 Introduction

Parnassus is a **pure-Python**, **fully GPU-compatible** framework for **fast detector simulation and reconstruction** in particle and nuclear physics. It provides an alternative path to computationally expensive full Geant4 [4] simulation and reconstruction chains with generative models that map truth-level (generator-level) particles directly to detector-level reconstructed particles, bypassing intermediate steps such as hit simulation, digitisation, and track reconstruction. The framework is both **process-agnostic** and **detector-agnostic**: users select a detector card to target a specific experiment, and the same tool can be applied to arbitrary physics processes by simply providing different input truth events.

Parnassus includes a number of interchangeable **generator backends** sharing a common API; these backends are of two types:

- A **neural backends** based on conditional flow matching [8, 9] trained on full simulation and reconstruction data, providing high-fidelity learned detector response.
- A **parametric backend**, TorchDelphes, which re-implements the Delphes [26] fast-simulation chain (particle propagation, tracking efficiency, momentum smearing, calorimeter response, energy-flow merging) in pure PyTorch. The implementation is validated against C++ Delphes 3.5.0 in Appendix A for both CMS and ATLAS detector cards and provides a GPU-native drop-in alternative to running C++ Delphes.

Unlike traditional fast simulation tools such as Delphes [26], which are written in C++, require the ROOT framework [20], and run only on CPU, Parnassus is implemented entirely in Python with PyTorch [23] and runs natively on GPU, enabling seamless integration into modern Python-based analysis workflows and significant acceleration on GPU hardware. ROOT I/O is supported via `uproot` [25] for writing output files, but is entirely optional — Parnassus has no dependency on ROOT itself.

Users select a **detector card** — analogous to choosing a detector card in Delphes — to target a specific experiment. The current release ships with CMS and ATLAS parametric cards and CMS (trained on 2011 open data [7]) and ALEPH neural models; additional detector models for other experiments will be added in future releases.

The physics methodology underlying Parnassus has been described in detail in Refs. [1, 2], where we demonstrated that the approach reproduces full CMS simulation and reconstruction across a wide range of Standard Model and BSM processes. In this document we focus on the **public software release** — its installation, usage, and API — to enable the broader community to adopt neural fast simulation and reconstruction in their workflows. Significant

software infrastructure has been built since the initial Parnassus demonstrations and this paper documents a milestone for the Parnassus project.

The neural backbone uses conditional flow matching [8, 9], a continuous-time generative modelling framework closely related to score-based diffusion models [10, 11], sampled via Euler integration of the learned vector field at inference time. The models are conditioned on truth-level particle properties, making the generation inherently process-agnostic without requiring retraining for new physics scenarios.

1.1 Design goals

Speed Orders of magnitude faster than full simulation and reconstruction by replacing the detector response and combinatorial pattern matching with a learned generative model. Fully GPU-compatible for additional acceleration.

Pure Python
Implemented entirely in Python with no compiled dependencies beyond PyTorch. Unlike C++/ROOT-based tools such as DELPHES [26], Parnassus integrates naturally into modern Python analysis ecosystems. ROOT output is available via `uproot` but is not required.

Fidelity Each neural detector model is trained on full simulation and reconstruction data to reproduce realistic detector-level distributions including particle kinematics, vertex positions, impact parameters, and particle identification. Excellent agreement with full simulation has been demonstrated across diverse SM and BSM processes [1–3].

Flexibility Accept truth-level input from any source — PYTHIA8 [12] on-the-fly generation (native plugin available), HepMC [13] files from external generators (MADGRAPH5_aMC@NLO SHERPA [15], HERWIG [16], POWHEG [17]), or custom formats. Users select a detector model to target a specific experiment, analogous to choosing a detector card in DELPHES.

Completeness
Full post-processing pipeline including jet clustering (FASTJET [18]) and lepton isolation, with optional output to ROOT format via `uproot` [25].

1.2 Output quantities

Given a set of truth-level particles for each event, Parnassus generates the quantities listed in Table 1.

Table 1: Output quantities produced by Parnassus for each event.

Output	Description
Reconstructed particles	Full kinematics (p_T, η, ϕ), vertex (v_x, v_y, v_z), class, PDG ID
Impact parameters	d_0, z_0 and their errors for charged particles
Jets	Clustered with configurable algorithms ; substructure, etc.
Lepton isolation	Isolation scores and p_T sums in configurable ΔR cones, etc.
Event-level quantities	H_T, $\vec{E}_\mathrm{T}^\mathrm{miss}$ components, particle multiplicity, etc.

2 Architecture overview

Parnassus passes truth-level events through a detector model followed by physics-based post-processing. The detector model can be either a neural backend or the TorchDelphes parametric backend; both share the same input and output interface so that downstream post-processing is unchanged. The pipeline is illustrated in Fig. 1.

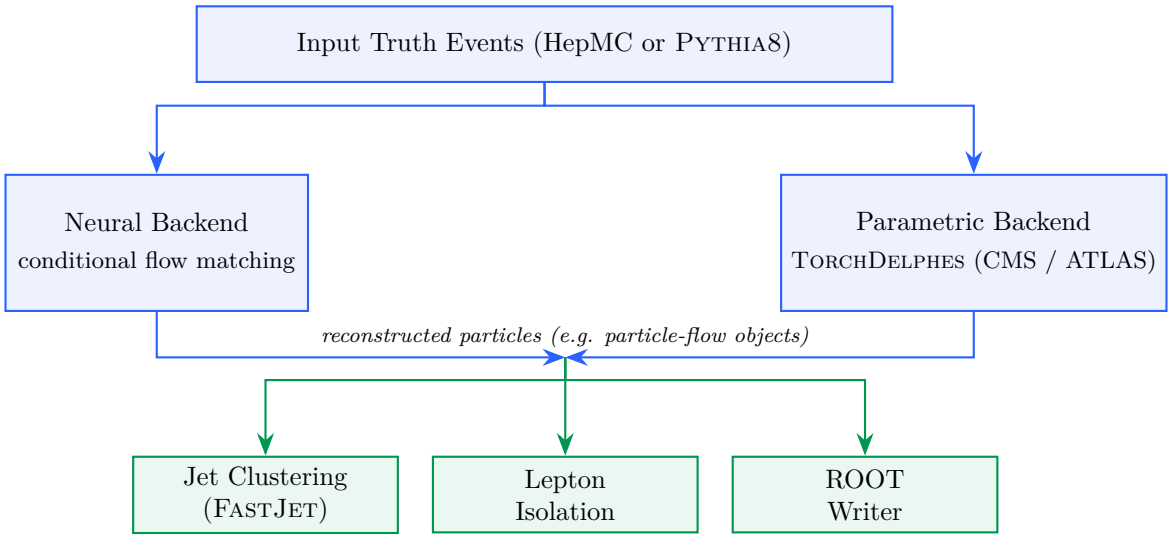


Figure 1: Parnassus generation pipeline. Truth-level events are routed through either the neural backend (conditional flow matching) or the parametric backend (TorchDelphes), both producing particle-flow objects. Physics-based post-processing (green) computes derived quantities and writes the final output.

The **neural backend** is a continuous-time generative model trained with conditional flow matching [8, 9] and sampled via Euler integration of the learned vector field. The model is conditioned on truth-level particle information, making the generation process inherently dependent on the input physics process without being tied to any specific one. The number of integration steps is configurable at runtime via the `--num_steps` (`-n`) flag, allowing users to trade generation speed for quality.

The **parametric backend**, TorchDelphes, is a pure-PyTorch re-implementation of the Delphes [26] fast-simulation chain. It models particle propagation through the magnetic field, tracking efficiency, momentum smearing, calorimeter response, and energy-flow merging, with per-module parameters loaded from the selected detector card. The current release provides CMS and ATLAS cards ported from their respective Delphes TCL cards and validated against C++ Delphes 3.5.0. Being implemented in pure PyTorch, TorchDelphes runs natively on GPU and shares the same input and output conventions as the neural backend.

Both backends expose the same output contract (particle-flow kinematics, vertex positions, particle classification, and, where supported, track impact parameters), so downstream post-processing is identical.

3 Installation and quick start

3.1 Installation

```
# Clone the repository
git clone https://github.com/parnassus-hep/parnassus.git
cd parnassus
```

```
# Install with pip (requires Python 3.12)
pip install .

# For development
pip install -e ".[dev]"
```

Listing 1: Installation commands.

3.2 Requirements

- Python $\geq$ 3.12
- PyTorch [23] $\geq$ 2.6.0
- Pythia8mc [12] 8.316.0
- PyHepMC [13] $\geq$ 2.14.0
- FastJet [18] $\geq$ 3.4.3.1
- EnergyFlow [24] $\geq$ 1.4.0
- Uproot [25] $\geq$ 5.5.2 *(optional, for ROOT output)*

3.3 Initialise a configuration file

```
parnassus init .
```

This copies the default `default_config.yaml` to the current directory. The default configuration uses the `cms_2011_flow_v00` neural detector model with anti-k_t jet clustering and lepton isolation. To target a different detector or to switch to the parametric (TorchDelphes) backend, change the `generator.type` and `generator.name` fields in the YAML configuration. The two generator backends are configured as shown in Listing 2.

```
# --- Neural backend (conditional flow matching) ---
generator:
  type: "neural"
  name: "cms_2011_flow_v00"  # trained CMS model
  num_steps: 50
  batch_size: 2000
  device: "cpu"              # "cpu", "cuda", or "mps"

# --- Parametric backend (TorchDelphes) ---
generator:
  type: "parametric"
  name: "cms"                # "cms" or "atlas" detector card
  seed: 42                   # optional, for reproducibility
```

Listing 2: Switching between the neural and parametric (TorchDelphes) generator backends.

3.4 Run with a HepMC input file

```
parnassus run \
  -c default_config.yaml \
  -i events.hepmc \
  -ne 1000 \
  -bs 2000 \
  -o output.root
```

Listing 3: Basic command-line invocation.

The available command-line flags are listed in Table 2.

Table 2: Command-line flags for `parnassus run`.

Flag	Long form	Description
`-c`	`--config`	Path to YAML configuration file
`-i`	`--input_path`	Input HepMC or Pythia `.cmnd` file
`-o`	`--output_path`	Output ROOT file path
`-ne`	`--num_events`	Number of events to process
`-bs`	`--batch_size`	Batch size for neural generation
`-n`	`--num_steps`	Neural-backend Euler integration steps (default: 50)

3.5 Run with Pythia8 on-the-fly generation

To generate truth-level events on-the-fly with Pythia8, create a `.cmnd` steering card and pass it as the input file. Parnassus detects `.cmnd` files automatically:

```
parnassus run \
  -c default_config.yaml \
  -i ttbar.cmnd \
  -ne 5000 \
  -bs 2000 \
  -o ttbar_fastsim.root
```

4 Demonstration: SM and BSM processes

The neural backend is **process-agnostic** — it learns the detector response as a function of truth-level particle properties, not the physics process itself — and the parametric (TorchDelphes) backend is process-agnostic by construction. We have previously demonstrated [1, 2] that Parnassus reproduces full simulation and reconstruction across a wide range of SM and BSM processes. Here we show two representative examples to illustrate the API in action: a Standard Model process ($H \to ZZ \to 4\ell$) and a BSM exotic Higgs decay ($H \to aa \to gg\mu\mu$). We then re-run the SM example with the parametric TorchDelphes backend to demonstrate that the two generator models can be swapped without touching the rest of the pipeline. The kinematic and class-fraction summaries that follow (Sections 4.5 onwards) are produced with the neural backend.

The current CMS neural model supports up to 400 truth particles per event. For events exceeding this limit, Parnassus retains the 400 highest-p_{T} particles so that no events are discarded.

4.1 Standard Model: $H \to ZZ \to 4\ell$

We process 100 events from a HepMC input file produced by Pythia8 [12]:

```
parnassus run \
  -c default_config.yaml \
  -i h4l_events.hepmc \
  -ne 100 \
  -o h4l_output.root
```

4.2 BSM: $H \to aa \to gg\mu\mu$

For the BSM example, we generate exotic Higgs decays on-the-fly with Pythia8. The Higgs decays to a pair of light pseudoscalars a ($m_a = 20\,\text{GeV}$), each decaying to either gg or $\mu^+\mu^-$, producing a mixed final state of jets and muon pairs:

```
! haa_ggmumu.cmnd -- BSM exotic Higgs decay
Beams:eCM = 13000.
Beams:idA = 2212
Beams:idB = 2212

! BSM Higgs production via gluon fusion
Higgs:useBSM = on
HiggsBSM:gg2H2 = on
35:m0 = 125.0                          ! Higgs mass

! SM-like couplings for production
HiggsH2:coup2u = 1.0
HiggsH2:coup2d = 1.0
HiggsH2:coup2A3A3 = 1.0                ! H -> aa coupling

! Force H -> a a only
35:onMode = off
35:onIfAll = 36 36

! Light pseudoscalar a properties (PDG ID 36)
36:m0 = 20.0                           ! mass = 20 GeV
HiggsA3:coup2d = 1.0
HiggsA3:coup2u = 1.0
HiggsA3:coup2l = 1.0

! Force a -> gg or mumu only
36:onMode = off
36:onIfAny = 21 13
```

Listing 4: Pythia8 steering card for BSM $H \to aa \to gg\mu\mu$.

```
parnassus run \
  -c default_config.yaml \
  -i haa_ggmumu.cmnd \
  -ne 1000 \
  -o haa_ggmumu_output.root
```

This demonstrates running Parnassus on a BSM signal process not present in the training data. The mixed $gg\mu\mu$ final state probes the model's ability to simultaneously generate hadronic jets and isolated muons within the same event.

4.3 Swapping backends: same events, parametric TorchDelphes

The two generator backends are swapped purely through the configuration file: neither the input HepMC file, the post-processing pipelines, nor the output schema change. Listing 5 shows a `cms_parametric.yaml` that re-runs the same $H \to ZZ \to 4\ell$ events through the parametric (TorchDelphes) backend with the CMS card.

```yaml
# cms_parametric.yaml
dataset:
  file_path: ""
  num_events: 100

# Swap only this block: neural -> parametric
generator:
  type: "parametric"    # was: "neural"
  name: "cms"           # was: "cms_2011_flow_v00"
  seed: 42

pipelines:
  TruthJetsAntiKt05: { type: cluster, collection: truth, dr: 0.5,
                       algorithm: antikt, pt_min: 10, nconst_min: 2 }
  PflowJetsAntiKt05: { type: cluster, collection: pflow, dr: 0.5,
                       algorithm: antikt, pt_min: 10, nconst_min: 2 }
  ElectronIsolation: { type: isolation, collection: electrons, dr: 0.4
      }
  MuonIsolation:     { type: isolation, collection: muons,     dr: 0.4
      }

output:
  file_path: ""
  format: default
```

Listing 5: Configuration for running TorchDelphes with the CMS card on the same $H \to 4\ell$ events.

```bash
parnassus run \
  -c cms_parametric.yaml \
  -i h4l_events.hepmc \
  -ne 100 \
  -o h4l_torchdelphes.root
```

Listing 6: Running the same events through the parametric backend.

The two output ROOT files (`h4l_output.root` from the neural backend and `h4l_torchdelphes.root` from TorchDelphes) follow the same schema, so the same downstream analysis code reads either of them unchanged. Switching to the ATLAS card requires only changing `generator.name` from `"cms"` to `"atlas"` in the same configuration file — analogous to selecting a different detector card in Delphes.

4.4 Event display

Fig. 2 shows a representative $H \to 4\ell$ event in the η–ϕ plane, illustrating how Parnassus transforms truth-level particles into detector-level pflow objects.

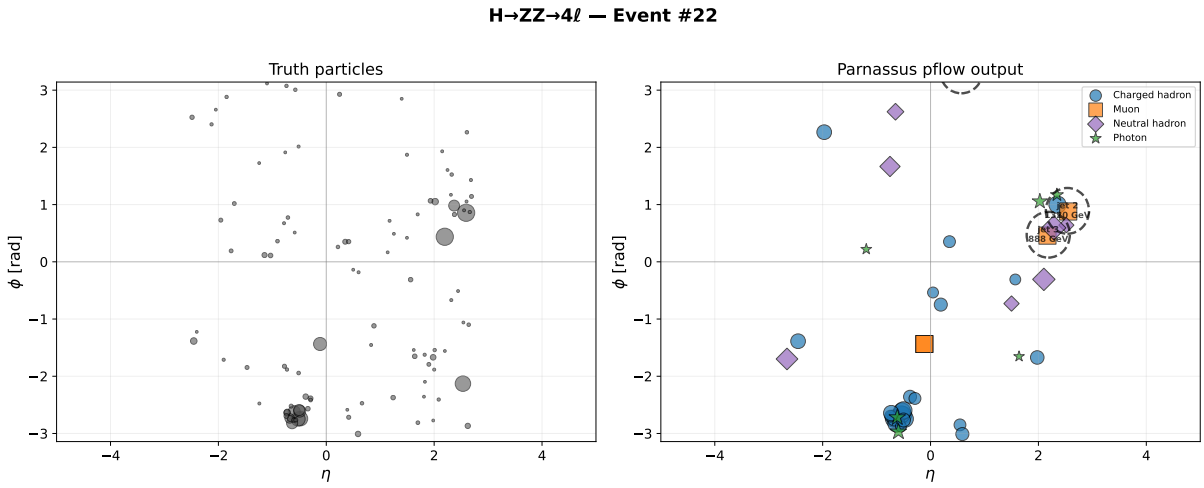


Figure 2: Event display for $H \to ZZ \to 4\ell$: truth particles (left) and Parnassus pflow output (right) in the η–ϕ plane. Marker size scales with p_{T}; dashed circles indicate anti-k_t jets ($R = 0.5$). Particle classes: charged hadrons (blue circles), electrons (red triangles), muons (orange squares), neutral hadrons (purple diamonds), photons (green stars). This plot is produced by `parnassus.utils.event_display`.

4.5 Process comparison

Fig. 3 compares jet-level and event-level observables for both processes using anti-k_t jets ($R = 0.5$, $p_{\mathrm{T}} > 10\,\mathrm{GeV}$). The BSM $H \to aa \to gg\mu\mu$ process produces hard jets from the $a \to gg$ decays, leading to markedly different dijet invariant mass (m_{jj}), jet multiplicity, leading-jet p_{T}, and H_{T} distributions compared to the purely leptonic SM $H \to ZZ \to 4\ell$ decay. In particular, the m_{jj} spectrum for the BSM process peaks near the pseudoscalar mass $m_a = 20\,\mathrm{GeV}$, while the SM process shows a broader distribution from underlying-event jets. These distributions demonstrate that the framework correctly propagates the physics differences between input processes into the reconstructed observables without any process-specific tuning.

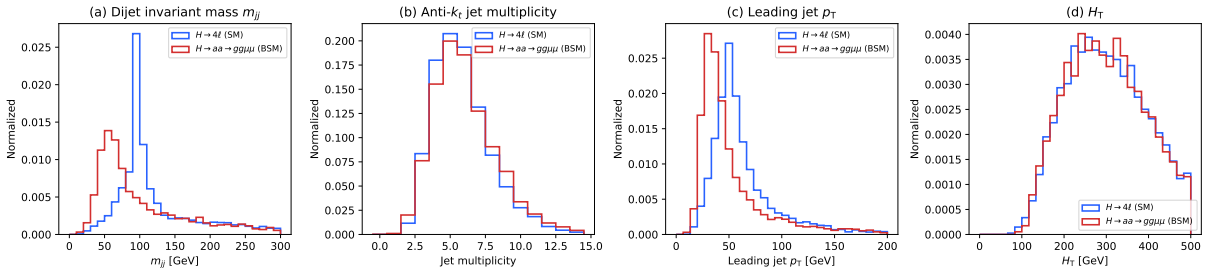


Figure 3: Jet-level and event-level distributions comparing SM ($H \to ZZ \to 4\ell$, 5000 events) and BSM ($H \to aa \to gg\mu\mu$, 5000 events) processes, using truth-level Pythia8 events clustered with anti-k_t jets ($R = 0.5$, $p_{\mathrm{T}} > 10\,\mathrm{GeV}$). (a) Dijet invariant mass m_{jj}, (b) jet multiplicity, (c) leading jet p_{T}, (d) scalar p_{T} sum H_{T}. All distributions are normalised to unit area. The script to reproduce this figure is included in `docs/prd/make_process_comparison.py`.

5 Python API

For programmatic usage, Parnassus provides a clean Python API:

```
from pathlib import Path
```

```python
from parnassus.configs import Config
from parnassus.pipelines import (
    GenerationPipeline,
    JetClusteringPipeline,
    IsolationPipeline,
)
from parnassus.configs.pipeline import (
    JetClusteringConfig,
    IsolationConfig,
)
from parnassus.writers import RootWriter

# Load configuration
config = Config.from_yaml("my_config.yaml")

# Override settings programmatically
config.dataset_config.file_path = (
    Path("my_events.hepmc").absolute()
)
config.dataset_config.num_events = 5000

# Run generation
pipeline = GenerationPipeline(config)
gen_events, accessors_dict = pipeline.run()

# Post-processing
for pipeline_config in config.pipeline_configs:
    if isinstance(pipeline_config, JetClusteringConfig):
        jet_pipeline = JetClusteringPipeline(pipeline_config)
        jet_pipeline.process(gen_events)
    elif isinstance(pipeline_config, IsolationConfig):
        iso_pipeline = IsolationPipeline(pipeline_config)
        iso_pipeline.process(gen_events)

# Inspect generated data
for event in gen_events[:5]:
    print(f"Event {event.event_number}:")
    n_truth = len(event.truth_particles.pt)
    n_pflow = len(event.pflow_particles.pt)
    print(f"  Truth particles: {n_truth}")
    print(f"  Pflow particles: {n_pflow}")
    print(f"  HT = {event.ht:.1f} GeV")

# Write to ROOT
writer = RootWriter(config.writer_config)
writer.write(gen_events)
```

Listing 7: Programmatic usage of the generation pipeline.

5.1 Reading output files

```python
import uproot
import numpy as np

f = uproot.open("output.root")
tree = f["Parnassus"]

```

```python
# Access pflow jet pT
jet_pt = tree["PflowJetsAntiKt05.pt"].array()

# Access electron isolation
e_iso = tree["Electrons.iso_var"].array()

# Access impact parameters
d0 = tree["Pflow.d0"].array()
z0 = tree["Pflow.z0"].array()
```

Listing 8: Reading the output ROOT file with `uproot`.

6 Configuration reference

```yaml
# -- Dataset ---------------------------------------
dataset:
  file_path: ""               # Path to .hepmc or .cmnd file
  num_events: 1000            # Number of events to process
  entry_start: 0              # Starting event index (HepMC)

# -- Generator (choose one backend) ------------------
# Option A: Neural backend (conditional flow matching)
generator:
  type: "neural"              # Select the neural backend
  name: "cms_2011_flow_v00"   # Detector model (like a Delphes card)
  num_steps: 50               # Number of flow-matching integration
      steps
  batch_size: 2000            # Events per batch
  device: "cpu"               # "cpu", "cuda", or "mps"

# Option B: Parametric backend (TorchDelphes)
# generator:
#   type: "parametric"        # Select the TorchDelphes backend
#   name: "cms"               # Detector card: "cms" or "atlas"
#   max_particles: 10000      # Maximum particles per event
#   seed: 42                  # Random seed for stochastic smearing

# -- Post-processing Pipelines -----------------------
pipelines:
  TruthJetsAntiKt05:
    type: "cluster"
    collection: truth
    dr: 0.5
    algorithm: antikt
    pt_min: 10
    nconst_min: 2
  PflowJetsAntiKt05:
    type: "cluster"
    collection: pflow
    dr: 0.5
    algorithm: antikt
    pt_min: 10
    nconst_min: 2
  ElectronIsolation:
    type: "isolation"
    collection: "electrons"
```

```
    dr: 0.4
  MuonIsolation:
    type: "isolation"
    collection: "muons"
    dr: 0.4

# -- Output ------------------------------------------
output:
  file_path: ""              # Output ROOT file path
  format: default
```

Listing 9: Annotated default configuration.

7 Conclusion

We have presented the public release of Parnassus, a pure-Python, GPU-compatible framework for fast simulation and reconstruction in High Energy Physics. Unlike traditional C++/ROOT-based tools such as DELPHES [26], Parnassus runs entirely in Python with PyTorch, supports GPU acceleration (CUDA and Apple MPS), and requires no ROOT installation — ROOT output is optionally handled via `uproot`. The software provides a simple command-line and Python API for generating detector-level particle-flow objects from truth-level input, with full post-processing including jet clustering and lepton isolation.

Parnassus offers two interchangeable generator backends that share a common user-facing API: a **neural backend** based on conditional flow matching models trained on full simulation, and a **parametric backend** (TORCHDELPHES) that re-implements the DELPHES fast simulation chain — particle propagation, tracking efficiency, momentum smearing, calorimetry, and energy-flow merging — as pure-PyTorch modules. TorchDelphes has been validated against C++ DELPHES 3.5.0 and provides a GPU-native drop-in alternative to C++ DELPHES.

The framework is both process-agnostic and detector-agnostic. Users select a detector model — analogous to choosing a detector card in DELPHES — to target a specific experiment. The current release ships with CMS and ATLAS parametric cards for the TorchDelphes backend, and a CMS neural model validated against full CMS simulation across diverse SM and BSM processes [1, 2]; additional detector models and neural checkpoints will be provided in future releases.

The source code is publicly available at https://github.com/parnassus-hep/parnassus under the MIT license.

References

[1] E. Dreyer, E. Gross, D. Kobylianskii, V. Mikuni, B. Nachman, and N. Soybelman, Parnassus: An Automated Approach to Accurate, Precise, and Fast Detector Simulation and Reconstruction, arXiv:2406.01620 (2024).

[2] E. Dreyer, E. Gross, D. Kobylianskii, V. Mikuni, and B. Nachman, Conditional Deep Generative Models for Simultaneous Simulation and Reconstruction of Entire Events, arXiv:2503.19981 (2025).

[3] Y. Lo, D. Kobylianskii, B. Nachman, An AI-based Detector Simulation and Reconstruction Model for the ALEPH Experiment at LEP, arXiv:2604.11834 (2026).

[4] S. Agostinelli et al. (GEANT4 Collaboration), Geant4 — a simulation toolkit, Nucl. Instrum. Meth. A **506** (2003) 250.

[5] CMS Collaboration, Particle-flow reconstruction and global event description with the CMS detector, JINST **12** (2017) P10003, arXiv:1706.04965.

[6] ATLAS Collaboration, Jet reconstruction and performance using particle flow with the ATLAS Detector, Eur. Phys. J. C **77** (2017) 466, arXiv:1703.10485.

[7] CMS Collaboration, CMS releases first batch of high-level LHC open data, CERN Open Data Portal (2014), http://opendata.cern.ch/.

[8] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, Flow Matching for Generative Modeling, ICLR (2023), arXiv:2210.02747.

[9] X. Liu, C. Gong, Q. Liu, Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, ICLR (2023), arXiv:2209.03003.

[10] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, B. Poole, Score-Based Generative Modeling through Stochastic Differential Equations, ICLR (2021), arXiv:2011.13456.

[11] J. Ho, A. Jain, P. Abbeel, Denoising Diffusion Probabilistic Models, NeurIPS (2020), arXiv:2006.11239.

[12] T. Sjöstrand et al., An introduction to PYTHIA 8.2, Comput. Phys. Commun. **191** (2015) 159, arXiv:1410.3012.

[13] A. Buckley et al., The HepMC3 event record library for Monte Carlo event generators, Comput. Phys. Commun. **260** (2021) 107310, arXiv:1912.08005.

[14] J. Alwall et al., The automated computation of tree-level and next-to-leading order differential cross sections, and their matching to parton shower simulations, JHEP **07** (2014) 079, arXiv:1405.0301.

[15] E. Bothmann et al. (Sherpa Collaboration), Event Generation with Sherpa 2.2, SciPost Phys. **7** (2019) 034, arXiv:1905.09127.

[16] J. Bellm et al., Herwig 7.0/Herwig++ 3.0 release note, Eur. Phys. J. C **76** (2016) 196, arXiv:1512.01178.

[17] S. Alioli, P. Nason, C. Oleari, E. Re, A general framework for implementing NLO calculations in shower Monte Carlo programs: the POWHEG BOX, JHEP **06** (2010) 043, arXiv:1002.2581.

[18] M. Cacciari, G. P. Salam, G. Soyez, FastJet User Manual, Eur. Phys. J. C **72** (2012) 1896, arXiv:1111.6097.

[19] M. Cacciari, G. P. Salam, G. Soyez, The anti-k_t jet clustering algorithm, JHEP **04** (2008) 063, arXiv:0802.1189.

[20] R. Brun and F. Rademakers, ROOT — An object oriented data analysis framework, Nucl. Instrum. Meth. A **389** (1997) 81–86, doi:10.1016/S0168-9002(97)00048-X.

[21] A. J. Larkoski, I. Moult, D. Neill, Power Counting to Better Jet Observables, JHEP **12** (2014) 009, arXiv:1409.6298.

[22] A. J. Larkoski, G. P. Salam, J. Thaler, Energy Correlation Functions for Jet Substructure, JHEP **06** (2013) 108, arXiv:1305.0007.

[23] A. Paszke et al., PyTorch: An Imperative Style, High-Performance Deep Learning Library, NeurIPS (2019), arXiv:1912.01703.

[24] P. T. Komiske, E. M. Metodiev, J. Thaler, Energy Flow Networks: Deep Sets for Particle Jets, JHEP **01** (2019) 121, arXiv:1810.05165.

[25] J. Pivarski et al., Uproot: ROOT I/O in pure Python and NumPy, https://github.com/scikit-hep/uproot5.

[26] J. de Favereau et al. (DELPHES 3 Collaboration), DELPHES 3: A modular framework for fast simulation of a generic collider experiment, JHEP **02** (2014) 057, arXiv:1307.6346.

A TorchDelphes

A.1 Implementation

DELPHES is composed of *modules*, each of which drives a different part of the detector emulation model. For example, the `Efficiency` class takes a set of particles as input and randomly masks a fraction of them according to a configurable efficiency map. One could configure the `Efficiency` class to represent the efficiency with which CMS tracks charged hadrons, and call this specific instantiation the `CMSChargedHadronTrackingEfficiency` module.

To emulate an entire detector such as ATLAS or CMS, users configure modules to represent each subsystem of their detector emulation model (e.g. electron momentum smearing, calorimeter hit clustering), and chain them together to define a computational graph for their intended simulation. This is done via a TCL configuration file; several are shipped pre-built with DELPHES, including the default ATLAS configuration `delphes_card_ATLAS.tcl` and the default CMS configuration `delphes_card_CMS.tcl`.

TORCHDELPHES defines PyTorch analogues of these C++ modules, providing GPU acceleration and a Python interface. Importantly, TORCHDELPHES does not implement *every* DELPHES module, only the modules necessary up to the generation of calorimeter tower hits ("Towers") and reconstructed photons, neutral hadrons, and charged tracks (collectively called "EFlowObjects"). Later post-processing that is independent of the actual detector interactions (e.g. jet clustering, object isolation) is handled by `parnassus.pipelines`, not TORCHDELPHES.

In addition to the constituent modules, TORCHDELPHES provides two pre-configured classes: `ATLASEnergyFlowDefault` and `CMSEnergyFlowDefault`, which correspond to the default ATLAS and CMS configurations that ship with DELPHES, respectively, again only up to the creation of Towers and EFlowObjects. These two default classes are used to run the validation suite.

A.2 Validation

We validate the TORCHDELPHES implementation by comparing its intermediate and final outputs against benchmarks generated by C++ DELPHES v3.5.1. We use the same HepMC inputs for both TORCHDELPHES and C++ DELPHES; these are generated by C++ PYTHIA8 v8.315. All code to run the full validation pipeline, including the configuration files to generate the HepMC inputs and the ROOT benchmarks, is available under `torch_delphes/validation/` in the repository.

We compare distributions generated by TORCHDELPHES against those generated by C++ DELPHES for the same HepMC inputs and detector configurations. We do this for both the ATLAS and CMS default detector configurations, and for two processes: $H \to ZZ \to 4\ell$ and $t\bar{t} \to W$.

For brevity, we limit this presentation to four particle-level kinematic variables: polar and azimuthal angles, momentum/energy, and transverse momentum/energy. We show both *final*

output object types — `Tower` and `EFlowObject`, which are generated stochastically — and one *intermediate* object type — `ParticleAfterPropagation`, which is generated deterministically. Users can validate the entire phase space (all particle-level distributions for all intermediate products) by passing the `--debug` flag to the validation script.[1]

As expected, there is perfect agreement for the deterministic outputs and very close agreement for the stochastic outputs, improving with sample size. Exact random-number matching between C++ DELPHES and TORCHDELPHES is not enforced, since the two implementations may run on different devices (CPU vs. GPU).

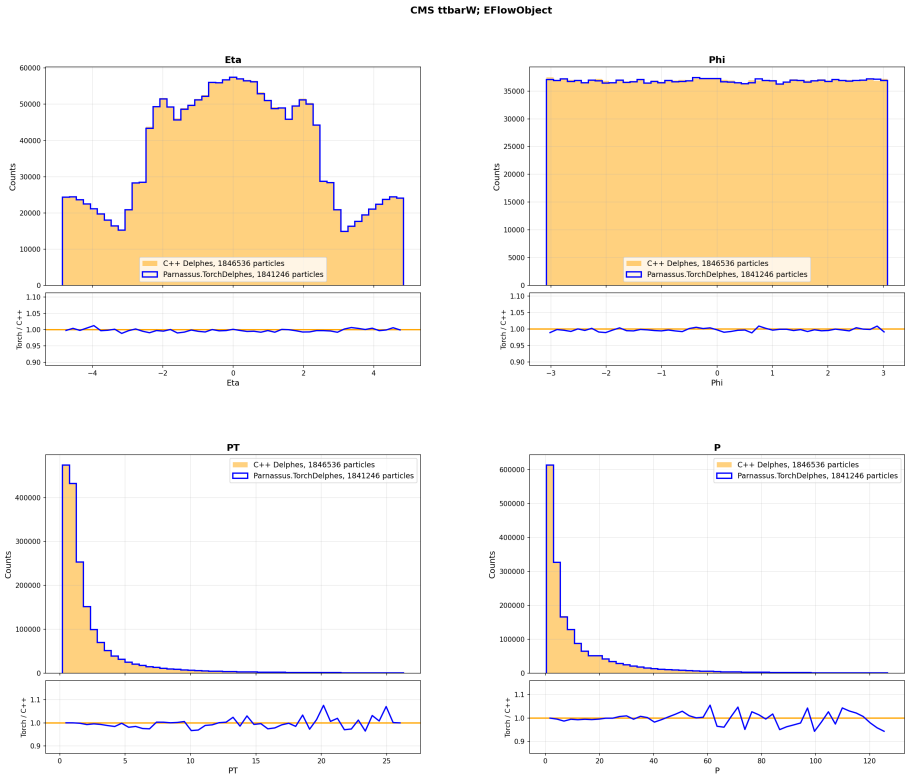
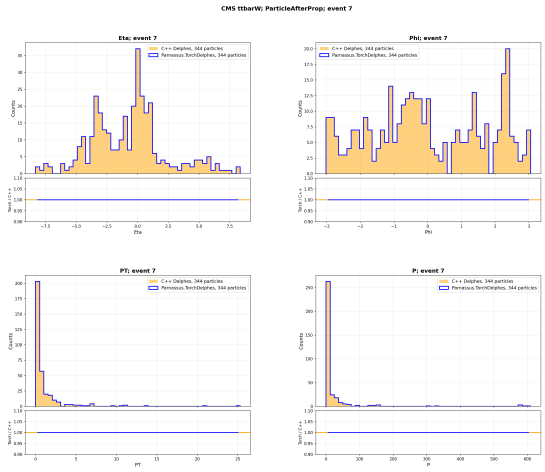
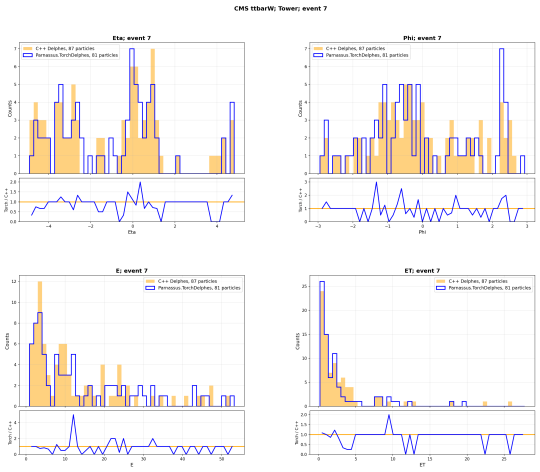


Figure 4: `EFlowObject` kinematics from 10,000 $t\bar{t} \to W$ input events via the default CMS detector configuration. Blue: C++ DELPHES; orange: TORCHDELPHES.

[1] See `torch_delphes/validation/README.md`.

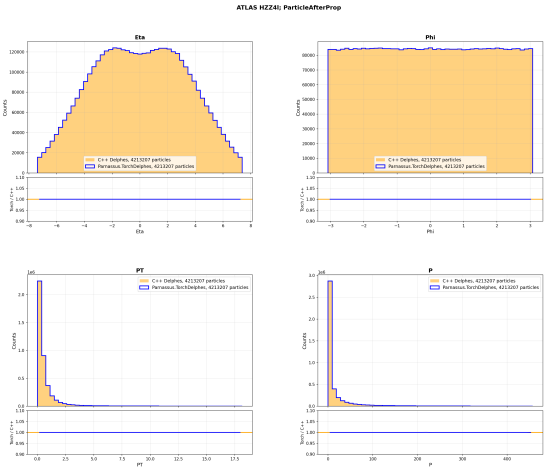


(a) The deterministically generated `ParticleAfterPropagation` objects show perfect agreement even on an event-by-event basis.

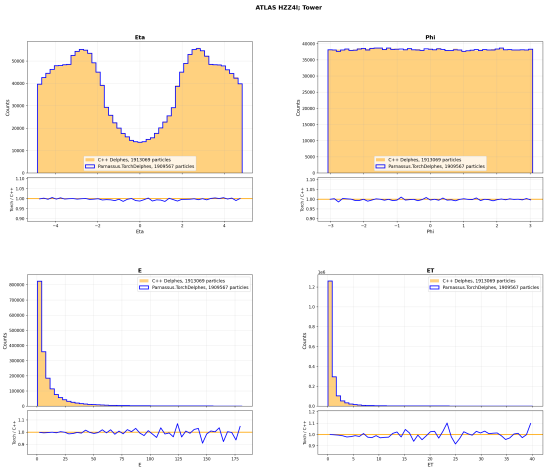


(b) The stochastically generated `Tower` objects show close but imperfect agreement due to independent random seeds.

Figure 5: Kinematics from a *single* $t\bar{t} \to W$ input event via the default CMS detector configuration.



(a) Perfect agreement for the deterministic `ParticleAfterPropagation` objects.



(b) Close agreement for the stochastic `Tower` objects; improved relative to the single-event comparison (Fig. 5b) due to increased statistics.

Figure 6: Kinematics from 10,000 $H \to ZZ \to 4\ell$ input events via the default ATLAS detector configuration.